

WOLLO University
Department of software engineering
Blueprint based exit material for
Data structure and algorithms

Prepared by:

@Neyatibeb

KOMBOLCHA, Ethiopia

2015 E.C

❖ Basic techniques for the design and analysis of efficient Algorithm

There are many different techniques for designing and analyzing efficient algorithms. Some of the most common techniques include:

- Divide-and-conquer:
 - It is a top-down approach
 - This technique breaks a problem down into smaller and smaller sub problems until they are trivial to solve. The solution to the original problem is then obtained by combining the solutions to the sub problems.
- Dynamic programming:
 - Bottom-up approach we solve all possible small problems and then combine them to obtain solutions for bigger problems.
 - This is particularly helpful when the number of copying subproblems is exponentially large. Dynamic Programming is frequently related to **Optimization Problems**.
 - This technique stores the solutions to sub problems so that they do not have to be recomputed repeatedly. This can be very efficient for problems that have overlapping sub problems.
- Greedy algorithms:
 - used to solve the optimization problem
 - These algorithms make a locally optimal choice at each step, hoping that this will lead to a globally optimal solution. Greedy algorithms are often simple to implement and can be very efficient for some problems.
- Amortized analysis:
 - This technique analyses the average-case running time of an algorithm, rather than the worst-case running time. This can be useful for algorithms that have occasional worst-case cases that can be very time-consuming.
- Randomization:
 - This technique uses randomness to improve the efficiency of an algorithm. Randomized algorithms can often be more efficient than deterministic algorithms, but they may not always be correct.
- **Branch and Bound:**
 - In Branch & Bound algorithm a given sub problem, which cannot be bounded, has to be divided into at least two new restricted sub problems. Branch and Bound algorithm are methods for global optimization in non-convex problems. Branch and Bound algorithms can be slow, however in the worst case they require effort that grows exponentially with problem size, but in some cases we are lucky, and the method coverage with much less effort.
- **Backtracking Algorithm:**
 - Backtracking Algorithm tries each possibility until they find the right one. It is a depth-first search of the set of possible solution. During the search, if an alternative doesn't work, then backtrack to the choice point, the place which presented different alternatives, and tries the next alternative.

The choice of which technique to use for a particular problem depends on the specific characteristics of the problem. Some problems are well-suited for divide-and-conquer, while others are better suited for dynamic programming. Greedy algorithms can be a good choice for problems where the optimal solution can be found by making a series of locally optimal choices. Amortized analysis can be used to analyse algorithms that have occasional worst-case cases. Randomization can be a good choice for problems where the optimal solution is difficult to find deterministically.

Department of software engineering

Here is an example of how one of these techniques can be used to design an efficient algorithm. The problem of finding the longest common subsequence of two strings is a classic problem in computer science. A dynamic programming solution to this problem works by storing the lengths of the longest common sub-sequences of all prefixes of the two strings. This information can then be used to quickly find the length of the longest common subsequence of the entire two strings.

The analysis of an algorithm is the process of determining how long it takes to run. The running time of an algorithm can be expressed in terms of the size of the input. For example, an algorithm that takes $O(n)$ time to run will run in time proportional to the size of the input, where n is the size of the input.

The analysis of an algorithm can be done using a variety of techniques. One common technique is to use the master theorem. The master theorem provides a way to analyse algorithms that use the divide-and-conquer technique. Another common technique is to use amortized analysis. Amortized analysis can be used to analyse algorithms that have occasional worst-case cases that can be very time-consuming.

The analysis of an algorithm is an important part of the design process. By analysing the running time of an algorithm, you can determine whether it is efficient enough for your needs.

❖ Develop appropriate algorithmic solutions to the problems

Some steps you can follow:

1. Understand the problem. What is the input? What is the output? What are the constraints?
2. Brainstorm possible solutions. What are some different ways to solve the problem?
3. Choose a solution. Consider the efficiency, accuracy, and simplicity of each solution.
4. Implement the solution. Write code or pseudocode to implement the solution.
5. Test the solution. Make sure the solution works correctly for all possible inputs.
6. Analyze the solution. Determine the time and space complexity of the solution.

Here are some examples of algorithmic solutions to problems:

- Sorting: There are many different sorting algorithms, such as bubble sort, selection sort, and merge sort. Each algorithm has its own strengths and weaknesses.
- Searching: There are also many different searching algorithms, such as linear search, binary search, and hash search. Each algorithm is suited for different types of problems.
- Graph problems: There are many different graph problems, such as finding the shortest path between two nodes, finding the maximum flow in a network, and finding the minimum spanning tree. Each problem has its own unique solution.
-

❖ Understand the principles of data storage and manipulation by applying data structures and algorithms

Data structures are a way of organizing data so that it can be efficiently stored and manipulated. There are many different data structures, each with its own strengths and weaknesses. Some common data structures include:

- Arrays: Arrays are a simple data structure that stores data in a linear order. Arrays are efficient for accessing data by index.

Department of software engineering

- **Linked lists:** Linked lists are a more complex data structure that stores data in a linked list. Linked lists are efficient for inserting and deleting data.
- **Stacks:** Stacks are a data structure that stores data in a last-in, first-out (LIFO) order. Stacks are efficient for implementing recursive algorithms.
- **Queues:** Queues are a data structure that stores data in a first-in, first-out (FIFO) order. Queues are efficient for implementing event-driven programs.

Algorithms are a set of instructions that can be used to solve a problem. There are many different algorithms, each with its own strengths and weaknesses. Some common algorithms include:

- **Sorting algorithms:** Sorting algorithms are used to sort data in a specific order. Some common sorting algorithms include bubble sort, selection sort, and merge sort.
- **Searching algorithms:** Searching algorithms are used to find a specific item in a data structure. Some common searching algorithms include linear search, binary search, and hash search.
- **Graph algorithms:** Graph algorithms are used to solve problems involving graphs. Some common graph algorithms include finding the shortest path between two nodes, finding the maximum flow in a network, and finding the minimum spanning tree.

The principles of data storage and manipulation can be understood by applying data structures and algorithms. Data structures provide a way to store data efficiently, while algorithms provide a way to manipulate data efficiently. By understanding these principles, you can design and implement efficient solutions to problems involving data.

Here are some examples of how data structures and algorithms can be used to store and manipulate data:

- A grocery store might use a linked list to store its inventory of products. This would allow the store to efficiently insert and delete products from its inventory.
- A bank might use a stack to store the transactions that it has processed. This would allow the bank to efficiently undo transactions that have been processed incorrectly.
- A social media website might use a queue to store the messages that its users have sent. This would allow the website to efficiently deliver messages to users in the order that they were sent.